



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Никола Огњеновић

**Имплементација погона за
дводимензионалне видео игре
користећи ЕКС архитектуру и
програмски језик Rust**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Никола Огњеновић	Број индекса:	SV 51 / 2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Имплементација погона за дводимензионалне видео игре користећи ЕКС архитектуру и програмски језик Rust
--

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Никола Огњеновић
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Имплементација погона за дводимензионалне видео игре користећи ЕКС архитектуру и програмски језик Rust
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	7/39/11/0/6/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Nikola Ognjenović
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	Implementation of a game engine for two-dimensional video games using ECS architecture and the Rust programming language
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/39/11/0/6/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Структура рада	1
2	Стање у области	3
2.1	Погони за видео игре	3
2.2	ЕКС парадигма	3
2.3	Савремени погони за видео игре	5
2.4	Разлози избора програмског језика <i>Rust</i>	6
3	Имплементација ЕКС језгра у <i>Rusty</i> погону	9
3.1	Преглед фајлова у <i>Rusty</i> погону	9
3.2	<code>lib.rs</code> : модулска декларација и јавни интерфејс	9
3.3	<code>entity.rs</code> : идентитет, генерације и рециклажа ентитета	10
3.4	<code>component.rs</code> : типизоване компоненте преко <i>type erasure</i> слоја	11
3.5	<code>event.rs</code> : систем догађаја са редовима по типу	14
3.6	<code>system.rs</code> : системи и извршни редослед	14
3.7	<code>world.rs</code> : интеграциони слој ЕКС језгра погона	15
3.8	Пример примене: текстуална борбена игра	16
4	Имплементација ECS-a у <i>rusty</i>	19
4.1	Ентитети као лагани идентификатори	19
4.2	Компоненте као чисти подаци	19
4.3	Структура <i>World</i> као центар координације	19
4.4	Складиште компоненти и регистар типова	20
4.5	Системи и догађаји	20
4.6	Пример из <i>text-game</i> пројекта	21
4.7	Поређење са алтернативним имплементацијама	21
4.8	Инжењерске импликације за даљи развој	21
5	Рендеринг подсистем и избор <i>wgri</i>	23
5.1	Улога рендеринга у укупној архитектури	23
5.2	<i>Renderer2D</i> као кључна апстракција	23
5.3	Основе <i>wgri</i> модела	23
5.4	Зашто <i>wgri</i> уместо алтернатива	24
5.5	Пут од ECS компоненти до <i>draw</i> позива	24
5.6	Шејдерски слој и ресурсни модел	24
5.7	Управљање редоследом исцртавања	25
5.8	Перформансне импликације	25
5.9	Ограничења тренутне имплементације	25
6	Рендеринг бенчмарк и евалуација перформанси	27
6.1	Циљ и улога бенчмарка	27
6.2	Конструкција сценарија	27
6.3	Механизам мерења FPS-a	27
6.4	Интерактивна контрола оптерећења	28
6.5	Предложена методологија спровођења мерења	28
6.6	Тумачење резултата	28
6.7	Веза са архитектурним одлукама	29
6.8	Ограничења бенчмарк приступа	29

6.9 Предлози за унапређење евалуације	29
7 Закључак	31
Списак коришћених скраћеница	33
Списак коришћених појмова	35
Биографија	37
Литература	39

Развој савремених видео игара подразумева изградњу сложених софтверских система који морају истовремено да задовоље захтеве флексибилности, перформанси и одрживости кода. Како пројекти расту, повећава се број међусобно повезаних подсистема, попут логики игре, управљања ресурсима, физичке симулације и рендеринга. Уколико архитектура није пажљиво осмишљена, временом долази до јаке спреге између различитих делова система, што отежава развој нових функционалности, тестирање и дугорочно одржавање пројекта.

Традиционални приступи засновани на дубоким хијерархијама класа дуго су представљали доминантан модел организације кода у развоју игара. Иако су погодни за мање пројекте, код сложенијих система често доводе до проблема као што су наслеђивање непотребних функционалности, тешкоће при поновној употреби кода и повећана зависност између компоненти. Као алтернатива таквом приступу, последњих година све већу популарност стиче *ЕКС* архитектура заснована на ентитетима, компонентама и системима (енг. *ECS, Entity Component System*), која омогућава јасније раздвајање података од логики и лакше скалирање сложених симулација.

У том контексту, овај рад се бави развојем погона за видео игре *Rusty*, заснованог на *ЕКС* архитектури и реализованог у програмском језику *Rust*. Полазна идеја пројекта била је да се испита у којој мери је могуће изградити функционалан *ЕКС* погон користећи искључиво безбедне механизме које *Rust* нуди, без ослањања на небезбедан (енг. *unsafe*) код. Поред провере техничке изводљивости таквог приступа, циљ је био и да се анализирају његове последице по архитектуру система, једноставност имплементације и перформансе.

Рад обухвата развој кључних делова погона и њихову примену у пратећим програмима. За потребе тестирања *ЕКС* језгра погона развијена је текстуална игра заснована на потезима, која демонстрира рад ентитета, компоненти и система у пракси. Поред тога, погон садржи рендеринг систем заснован на технологији *wgpu*, који омогућава приказ дводимензионалне сцене коришћењем савремених графичких *АПИ*-ја (апликациони програмски интерфејс, енг. *application programming interface, API*). У оквиру пројекта реализован је и програм *rendering-benchmark*, чија је сврха испитивање граница система кроз сценарије масовног креирања ентитета, анализу њиховог утицаја на брзину извршавања и праћење вредности броја кадрова у секунди (енг. *frames per second, FPS*).

Кроз наредна поглавља биће представљени теоријски концепти на којима је заснован развој погона, донете архитектонске одлуке, имплементациони детаљи, као и резултати евалуације који омогућавају сагледавање предности и ограничења предложеног решења.

1.1 Структура рада

Рад је подељен у више поглавља, почевши од стања у индустрији, преко теоријских основа, до практичне реализације и евалуације:

- у другом поглављу дат је преглед стања у индустрији што се тиче погона за видео игре, њихових врста и тренутно популарних алата, описана је *ЕКС* архитектура, њене предности у односу на алтернативне приступе и разлози због којих је *Rust* изабран као програмски језик за развој погона;
- у трећем поглављу описана је *ЕКС* архитектура погона за видео игре *Rusty*;
- у четвртном поглављу представљене су могућности имплементације рендеринга у погон за видео игре и образложен избор библиотеке *wgpu*;

- у петом поглављу описани су рендеринг систем, детаљи рада са *wgpu* библиотеком и прелаз из *ЕКС* података ка представи на графичком процесору;
- у шестом поглављу приказани су дизајн бенчмарка, методологија мерења и интерпретација добијених перформансних показатеља;
- у седмом, закључном поглављу, сумирани су резултати рада, размотрена ограничења система и предложени могући правци даљег развоја *Rusty* погона за видео игре.

2.1 Погони за видео игре

Погон за видео игре (енг. *game engine*) представља софтверски систем који обезбеђује основну инфраструктуру за развој видео игара. Уместо да програмер за сваку игру изнова имплементира рендеринг, управљање ресурсима, обраду улаза, физику, аудио систем и организацију логике игре, погон пружа унификован скуп механизма који се могу поново користити.

Савремени погони за видео игре представљају слој апстракције између оперативног система и апликативне логике игре. Они обједињују више подсистема који заједно омогућавају извршавање игре у реалном времену. Типичан погон обухвата:

- систем за рендеровање графике,
- систем за читавање и управљање ресурсима,
- управљање сценом,
- обраду корисничког улаза,
- аудио подсистем,
- систем за физику и колизије,
- скриптовање или програмски интерфејс,
- алате за развој и дебаговање.

Историјски посматрано, раније видео игре често нису користиле издвојене погоне. Логика игре и инфраструктурни код били су тесно повезани у једној апликацији. Како су пројекти постајали сложенији, појавила се потреба за поновном употребом истих механизма кроз више различитих игара. Из тог разлога постепено се формира концепт погона као засебног слоја који омогућава раздвајање доменске логике игре од инфраструктурних компоненти.

Данас су погони за видео игре један од централних елемената индустрије јер значајно смањују време развоја и омогућавају тимовима да се фокусирају на дизајн и имплементацију конкретне игре уместо на поновно решавање истих техничких проблема.

2.2 ЕКС парадигма

ЕКС представља архитектонски образац који је постао широко прихваћен у модерном развоју видео игара. Основна идеја ЕКС-а јесте раздвајање идентитета објеката, њихових података и логике која те податке обрађује.

ЕКС организује систем у три основна слоја:

- ентитети представљају идентификаторе,
- компоненте представљају податке,
- системи представљају логику обраде.

Ентитет обично не садржи никакво понашање нити податке. Он представља само идентификатор над којим се повезују компоненте.

Компоненте су структуре података које описују одређен аспект ентитета. На пример, позиција, брзина, здравље или визуелни приказ могу бити засебне компоненте.

Системи извршавају алгоритме над групама компоненти. Систем за кретање може истовремено обрађивати све ентитете који поседују компоненте позиције и брзине, без потребе да зна било шта о осталим аспектима тих ентитета.

Према дефиницији ЕКС архитектуре, она је заснована на принципу композиције уместо наслеђивања и омогућава да се понашање објекта формира комбиновањем компоненти уместо креирањем дубоких хијерархија класа [1].

2.2.1 Историјски развој ЕКС приступа

Идеје које стоје иза ЕКС архитектуре појављују се постепено током развоја игара почетком две хиљадитих година.

Као један од значајних раних извора често се наводи текст “*Evolve Your Hierarchy*” Ричарда Лорда из 2007. године који критикује дубоке хијерархије наслеђивања и предлаже композициони приступ организацији објеката у играма [1].

Истраживања ЕКС приступа постају интензивнија са растом потребе за већим бројем објеката у сцени и бољим искоришћењем кеш меморије процесора. Истовремено се развија и *data-oriented* дизајн као приступ који организацију података ставља у први план приликом пројектовања софтвера.

У литератури се ЕКС често повезује управо са *data-oriented* приступом јер раздвајање компоненти у хомогене скупове омогућава ефикасније секвенцијално приступање меморији и мањи број кеш промашаја [2].

2.2.2 Зашто се користи ЕКС

Главни разлози због којих се ЕКС користи у развоју видео игара су:

- већа флексибилност композиције понашања,
- мања повезаност између делова система,
- лакше проширење функционалности,
- погодност за паралелну обраду,
- боље искоришћење процесорског кеша,
- једноставније управљање великим бројем ентитета.

Уместо да сваки објекат садржи комплетну логику и стање, системи обрађују хомогене скупове података. То значајно поједностављује додавање нових функционалности јер је често довољно дефинисати нову компоненту или систем без измене постојећих структура.

2.2.3 ЕКС погони и библиотеке

Поред *Unity DOTS* екосистема, данас постоји велики број ЕКС решења.

Најзначајнији представници су:

- *Unity ECS*,
- *Bevy*,
- *flecs*,
- *EnTT*,
- *Legion*,
- *Shipyard*.

Unity ECS представља интегрисано решење у оквиру *DOTS* екосистема.

Bevy користи ЕКС као централни архитектонски концепт и представља један од најзначајнијих примера примене ЕКС-а у *Rust* екосистему.

EnTT и *flecs* представљају популарне C++ ЕКС библиотеке које се често користе приликом израде сопствених погона за видео игре.

2.2.4 Поређење ЕКС приступа са алтернативама

Најчешћа алтернатива ЕКС приступу је класичан објектно оријентисан модел.

У класичном моделу објекат истовремено садржи и стање и понашање. Како број различитих типова објеката расте, хијерархије класа постају све дубље и сложеније.

Типичан пример је ситуација у којој постоје класе:

- *GameObject*,
- *Character*,
- *Enemy*,
- *FlyingEnemy*,
- *BossEnemy*.

Временом се појављује велики број комбинација понашања које је тешко изразити искључиво наслеђивањем.

ЕКС приступ избегава овај проблем јер омогућава да се понашање формира композицијом компоненти. Ентитет који има компоненте *Transform*, *Renderable* и *Health* може представљати један тип објекта, док ентитет који има *Transform*, *Renderable*, *Health* и *AI* представља други, без потребе за новим нивоом наслеђивања.

За архитектуралну основу *Rusty* погона изабран је ЕКС јер:

- раздваја податке и логику,
- омогућава лако проширење система,
- поједностављује тестирање,
- погодан је за *data-oriented* организацију података,
- природно се уклапа у *Rust* модел власништва и позајмљивања.

2.3 Савремени погони за видео игре

У пракси се могу издвојити три основне групе решења:

- комерцијални општенаменски погони,
- отворени погони и радни оквири,
- наменски или интерни погони.

Комерцијални погони најчешће представљају комплетна решења која укључују визуелне едиторе, системе за изградњу пројекта, интегрисане алате за профилисање и велики број готових пакета.

Отворени погони су често усмерени ка већој флексибилности и могућности измене изворног кода. Они су значајни и у академском контексту јер омогућавају анализу архитектуре и експериментисање са различитим приступима.

Интерни погони настају у оквиру студија који развијају игре специфичних захтева. Такви системи су обично прилагођени конкретном жанру или типу пројекта и често не морају да решавају све проблеме које покривају општенаменски погони.

Најзначајнији представници данашње индустрије су *Unity*, *Unreal Engine*, *Godot* и *Bevy*.

2.3.1 *Unity*

Unity је један од најраспрострањенијих погона у индустрији. Његова популарност произилази из релативно ниске улазне баријере, великог екосистема пакета, документације и мултиплатформске подршке.

Током већег дела свог развоја *Unity* је био заснован на моделу *GameObject + MonoBehaviour*, где су објекти сцене организовани кроз компоненте везане за хијерархију објеката. Са растом захтева за перформансама и већим бројем ентитета на сцени, *Unity* је увео *DOTS (Data-Oriented Technology Stack)* који уводи ЕКС приступ и *data-oriented* дизајн [3], [4].

2.3.2 Unreal Engine

Unreal Engine представља један од најзначајнијих индустријских стандарда, посебно у сегменту комплексних тродимензионалних игара и AAA продукције.

Архитектура је заснована на *Gameplay Framework*-у где су учесник (енг. *Actor*) и компоненте основне јединице композиције [5]. Иако овај приступ није ЕКС у строгом смислу, он решава сличан проблем избегавања дубоких хијерархија наслеђивања кроз композицију функционалности.

2.3.3 Godot

Godot је отворен и општенаменски погон који користи архитектуру засновану на чворовима (енг. *node-based architecture*). Сваки објекат у сцени представљен је чвором који може имати подређене чворове и специфичну функционалност.

Овакав приступ омогућава једноставно организовање логике сцене, али се архитектонски разликује од ЕКС приступа јер је структура система оријентисана ка хијерархији чворова уместо ка обради хомогених скупова података.

2.3.4 Bevy

Bevy је савремени *Rust* погон чија је архитектура од почетка пројектована око ЕКС парадигме. У опису пројекта наглашава се да је реч о *data-oriented* погону заснованом на ЕКС моделу [6], [7].

Због директног ослањања на *Rust* екосистем и *data-oriented* приступ, *Bevy* представља значајну референтну тачку за радове који истражују развој погона за видео игре у програмском језику *Rust*.

2.4 Разлози избора програмског језика Rust

Циљ овог рада је доказати да ли је могуће реализовати функционалан ЕКС погон и рендеринг ток без употребе небезбедног кода у апликативном слоју.

Из тог разлога је за израду погона изабран програмски језик *Rust*.

2.4.1 Меморијска безбедност

Rust обезбеђује меморијску безбедност кроз систем власништва (енг. *ownership*), позајмљивања (енг. *borrowing*) и животних векова (енг. *lifetimes*).

За разлику од многих системских језика, велики број грешака може бити откривен већ током компилације. То укључује:

- коришћење ослобођене меморије,
- двоструко ослобађање,
- невалидне мутабилне референце,
- део конкурентних грешака и *data race* ситуација.

Ове гаранције су посебно значајне у развоју погона за видео игре јер се велики број подсистема истовремено ослања на исте структуре података.

2.4.2 Поређење са C и C++ приступом

Традиционално су многи погони за видео игре развијани у програмском језику C или C++.

У C приступу програмер има потпуну контролу над меморијом, али је истовремено одговоран за исправност свих операција које се тичу управљања ресурсима.

Такав модел омогућава висок ниво флексибилности, али повећава ризик од грешака које је често тешко репродуковати и анализирати.

При развоју *ЕКС* система у *С* језику значајан део времена може бити потрошен на:

- праћење животног века података,
- дебаговање меморијских грешака,
- анализу корупције меморије,
- синхронизацију приступа дељеним структурама.

Rust помера велики део ових провера у фазу компилације. Последица тога је да се сложеност јавља раније у процесу развоја, током обликовања архитектуре, уместо касније током дебаговања.

За пројекат као што је *Rusty* то представља значајну предност јер омогућава да се више времена посвети организацији *ЕКС* архитектуре и анализи перформанси, уместо отклањању нисконивоских меморијских проблема.

2.4.3 Улога безбедног и небезбедног *Rust*-а

Rust није ограничен искључиво на безбедни подскуп језика. Поред механизма који омогућавају статичку проверу великог броја безбедносних својстава програма, језик пружа и могућност употребе небезбедног (енг. *unsafe*) блока за операције чију исправност компајлер не може у потпуности да верификује.

Према званичној документацији језика, постојање механизма за небезбедне блокове кода произилази из чињенице да је статичка анализа по својој природи конзервативна, односно да не може увек са сигурношћу да докаже исправност свих легалних програма [8].

Употреба небезбедних блокова кода не значи искључивање правила језика, већ представља начин да програмер експлицитно преузме одговорност за одржавање одређених инваријанти које компајлер не може самостално да провери [8].

2.4.4 Могућност развоја погона без небезбедног кода

Циљ овог рада није доказивање да потпун спој графичких библиотека може бити без небезбедног кода. Библиотеке нижег нивоа, попут графичких апстракција или интерфејса према оперативном систему, често користе небезбедне механизме.

Предмет анализе је апликативни и архитектурални слој погона.

Основна хипотеза рада гласи да је могуће развити:

- *ЕКС* архитектуру,
- систем компоненти,
- систем ентитета,
- организацију рендеринг тока,
- управљање ресурсима,

без увођења небезбедних сегмената у доменској логици погона.

Практична реализација *Rusty* погона показује да је такав приступ изводљив за прост дводимензионални *ЕКС* погон чији су приоритети јасна архитектура, модуларност, предвидиво управљање ресурсима и разумљив модел проширења функционалности.

Због тога је *Rust* представљао природан избор за овај рад, јер омогућава комбинацију системског нивоа контроле, модерних апстракција и меморијске безбедности без потребе за сакупљачем отпадака (енг. *garbage collector*).

Глава 3

Имплементација ЕКС језгра у Rusty погону

Ово поглавље детаљно описује како је ЕКС архитектура имплементирана у оквиру библиотеке Rusty.

За разлику од претходног поглавља, које даје теоријски и компаративни оквир ЕКС приступа, овде је фокус на конкретним структуралним одлукама у коду: организацији ентитета, складиштењу компоненти, догађајном механизму, извршавању система и интеграцији тих делова у централну структуру света (енг. *world*).

Код је организован модуларно и прати принцип раздвајања одговорности који је карактеристичан за *data-oriented* пројектовање [2].

3.1 Преглед фајлова у Rusty погону

ЕКС језгро погона ослања се на следеће изворне фајлове:

- `lib.rs` - улазна тачка библиотеке, дефинише модуле и јавни АПИ;
- `entity.rs` - модел ентитета и управљање животним циклусом;
- `component.rs` - апстракције компоненти, складишта и менаџер компоненти;
- `event.rs` - догађајни модел и вишетипски догађајни редови;
- `system.rs` - дефиниција система и секвенцијални извршилац;
- `world.rs` - интеграциони слој који обједињује ентитете, компоненте и догађаје, уз могућност серијализације стања.

Ради провере исправности имплементације, сваки модул је засебно тестиран јединичним тестовима дефинисаним унутар одговарајућег изворног фајла употребом атрибута `#[cfg(test)]`. На овај начин омогућена је изолована верификација функционалности ентитета, компоненти, догађаја, система и интеграционог слоја пре њиховог повезивања у јединствену целину.

3.2 `lib.rs`: модулска декларација и јавни интерфејс

Фајл `lib.rs` прво излаже унутрашње модуле (`entity`, `component`, `event`, `world`, `system`, као и `render`), а затим реекспортује централне типове (`Entity`, `World`, `SystemExecutor` и друге) тако да корисник библиотеке може да ради са једноставним и конзистентним АПИ-јем.

```
pub mod entity;
pub mod component;
pub mod event;
pub mod world;
pub mod system;

pub use entity::{Entity, EntityManager};
pub use component::{Component, ComponentManager, HashMapComponentStorage};
pub use event::{Event, EventManager, EventQueue};
pub use world::World;
pub use system::{System, SystemExecutor};
```

Кључна предност реекспортовања централних типова кроз коренски модул библиотеке јесте стабилност јавног АПИ-ја. Кориснички код зависи од нпр. `rusty::World` или `rusty::Entity`, а не од дубоке интерне путање модула. Практично, то смањује трошак каснијих реорганизација изворног кода, јер унутрашње промене могу да остану транспарентне за корисника ако се спољни АПИ не мења.

Алтернатива је била да се типови не реекспортују, већ да сваки клијент увози симболе из подмодула. Тај приступ је формално исправан, али води ка дужим use путањама и јачој спрези корисника са унутрашњом структуром библиотеке.

3.3 entity.rs: идентитет, генерације и рециклажа ентитета

Модул entity.rs дефинише две централне структуре: Entity и EntityManager.

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Serialize, Deserialize)]
pub struct Entity {
    pub id: u32,
    pub generation: u32,
}
```

Entity је реализован као пар (id, generation), где је id идентификатор, а generation генерација тог идентификатора. Ово је познат образац за решавање проблема „висећих“ референци на обрисане ентитете у ECS системима [1].

EntityManager одржава:

- next_id за доделу нових идентификатора,
- free_ids као листу слободних идентификатора за поновну употребу,
- generations као вектор у коме сваки индекс директно одговара једном идентификатору, а вредност на тој позицији представља тренутну генерацију тог идентификатора.

```
#[derive(Serialize, Deserialize, Clone)]
pub struct EntityManager {
    next_id: u32,
    free_ids: Vec<u32>,
    generations: Vec<u32>,
}
```

create метода креира нови ентитет тако што прво покушава да рециклира слободан идентификатор из листе free_ids, а ако га нема, додељује нови идентификатор и иницијализује његову генерацију са вредношћу 0.

```
pub fn create(&mut self) -> Entity {
    if let Some(id) = self.free_ids.pop() {
        Entity { id, generation: self.generations[id as usize] }
    } else {
        let id = self.next_id;
        self.next_id += 1;
        self.generations.push(0);
        Entity { id, generation: 0 }
    }
}
```

destroy метода означава ентитет као неважећи тако што проверава да ли се генерација поклапа са тренутном, затим повећава генерацију и враћа id у free_ids ради поновне употребе.

```
pub fn destroy(&mut self, entity: Entity) {
    if (entity.id as usize) < self.generations.len() && self.generations[entity.id as usize]
    == entity.generation
    {
        self.generations[entity.id as usize] += 1;
        self.free_ids.push(entity.id);
    }
}
```

3.3.0.1 Зашто не забранити рециклажу идентификатора?

Потпуна забрана поновне употребе идентификатора делује безбедно, али у пракси доводи до исцрпљивања свих могућих идентификатора у дуготрајним симулацијама. Уместо тога, рециклажа идентификатора уз употребу генерација пружа ограничен и стабилан простор идентификатора, уз задржавање безбедности референци.

3.3.1 Зашто генерације, а не само идентификатор

Када би ентитет био моделован само као `id: u32` без генерације, појавио би се проблем рециклирања идентификатора:

1. ентитет са `id = 5` се обрише,
2. исти `id = 5` се касније додели новом ентитету,
3. стари референцирани ентитет и даље постоји у системима и сада погрешно указује на нови ентитет.

Комбинација идентификатора и генерације решава овај проблем тако што сваки животни циклус истог идентификатора добија нову верзију. Стари (5, 0) и нови (5, 1) више нису упоредиви као исти ентитет, што омогућава детерминистичку проверу валидности референци.

3.3.1.1 Зашто не један u64?

Иако би се идентификатор и генерација могли спаковати у једну 64-битну вредност, овај приступ у ЕКС архитектурама има неколико недостатака. Пре свега, он уклања експлицитну структурну разлику између идентификатора и верзије, што смањује читљивост и отежава логичко расуђивање о систему.

Поред тога, одвојено чување `id` и `generation` често омогућава боље кеш понашање у *data-oriented* дизајну, јер системи могу независно да обрађују идентификаторе и метаподатке о верзијама.

3.3.1.2 Зашто не UUID?

Коришћење *UUID* (енг. *Universally Unique Identifier*) вредности обезбеђује глобалну јединственост, али за прост ЕКС систем уводи значајан меморијски и перформансни трошак. *UUID* обично заузима 128 бита, што је четири пута више од 32-битног идентификатора, уз додатно погоршање кеш локалности приликом масовне обраде ентитета.

UUID је погоднији за дистрибуиране системе и перзистентне ресурсе, али мање ефикасан у високофреквентним симулацијама са великим бројем краткоживећих објеката.

3.3.1.3 Инжењерски компромис

У пракси, модел заснован на пару `id` и `generation` представља компромис између перформанси, меморијске ефикасности и безбедности. Он омогућава:

- $O(1)$ приступ ентитетима,
- ефикасно коришћење меморијског простора,
- детекцију застарелих референци без глобалне координације.

Овај приступ је широко прихваћен у савременим ЕКС системима, попут *Bevy* [9].

3.4 `component.rs`: типизоване компоненте преко *type erasure* слоја

`component.rs` је најобимнији део погона и садржи комплетну инфраструктуру за складиштење компоненти различитих типова. Сваки тип компоненте има своје засебно складиште, чиме се обезбеђује типска изолација података и избегава мешање различитих структура у меморији.

3.4.1 *Trait Component*

`Component` је *marker trait* који дефинише скуп ограничења која свака компонента мора да испуњава. Овај *trait* представља заједнички именитељ који омогућава да сва различита складишта компоненти буду третирана на унифициран начин у оквиру система.

Конкретно, компонента мора да:

- имплементира Any, чиме се омогућава идентификација типа у време извршавања,
- има 'static животног века, што омогућава безбедно коришћење кроз механизме као што је Any.
- подржава серијализацију и десеријализацију путем Serialize и Deserialize,

Ова ограничења омогућавају да се типови компоненти динамички препознају током извршавања програма (коришћењем TypeId или Any), уместо да се ослањају искључиво на статичку типску проверу у време компајлирања.

```
pub trait Component: Any + Serialize + for<'de> Deserialize<'de> + 'static {}
```

```
impl<T: Any + Serialize + for<'de> Deserialize<'de> + 'static> Component for T {}
```

Ограничење на Serialize / Deserialize није универзални захтев свих EKC библиотека, али је у Rusty уведено јер World подржава чување и учитавање стања у JSON формату преко serde екосистема [10], [11].

Пошто сваки тип компоненте има своје засебно складиште, јавља се потреба да се та хетерогеност сакрије иза јединственог интерфејса.

3.4.2 Брисање типа складишта компоненти и динамички диспеч

У Rust-у, типски систем по правилу захтева да конкретни типови буду познати у време компајлирања. Међутим, у EKC архитектури, ComponentManager у једној структури управља складиштима различитих компоненти (нпр. Position, Velocity, Health и сл.), при чему свака од њих има различит тип.

Да би се ово омогућило, користи се техника брисања типа (енг. *type erasure*), где се конкретни типови „скривају“ иза заједничког интерфејса dyn ComponentStorage. На тај начин, различита складишта могу да се чувају у истој колекцији без познавања њиховог конкретног типа у тренутку компајлирања.

Овако добијени апстрактни слој омогућава да се сва складишта индексирају по TypeId, чиме се формира централни регистар свих компоненти у систему.

Конкретан тип се поново „открива“ тек када је потребно, коришћењем механизма downcast-a (downcast_ref / downcast_mut). Ова операција представља контролисано враћање са динамичког типа на конкретан тип, али само у случају када је стварни тип у складу са очекиваним.

```
pub trait ComponentStorage: Any {
    fn as_any(&self) -> &dyn Any;
    fn as_any_mut(&mut self) -> &mut dyn Any;
    fn remove(&mut self, entity: Entity);
}
```

У наставку се види пример враћања конкретног складишта за одређени тип T:

```
pub fn get_storage_by_type<T: Component>(&self) -> Option<&HashMapComponentStorage<T>> {
    self.storages
        .get(&TypeId::of::<T>())?
        .as_any()
        .downcast_ref::<HashMapComponentStorage<T>>()
}
```

Другим речима, јавни интерфејс остаје потпуно статички типски безбедан (нпр. get_component::<T>), док се унутрашња имплементација ослања на динамички диспеч (енг. *dynamic dispatch*) како би се омогућила хетерогена колекција складишта у једном систему.

Иако је приступ складиштима унификован, унутрашња репрезентација сваког складишта остаје специфична за конкретан тип компоненте.

3.4.3 HashMapComponentStorage<T: Component> и ComponentManager

Конкретна имплементација складишта је HashMapComponentStorage<T>, где је унутрашња структура HashMap<Entity, T>. Зато је приступ „ентитет -> компонента“ просечно $O(1)$, уз очекиване карактеристике хеш структура.

ComponentManager има два речника:

- `storages: HashMap<TypeId, Box<dyn ComponentStorage>>`,
- `type_names: HashMap<String, TypeId>`.

Први речник служи за приступ складиштима по типу током извршавања кода, док други уводи стабилно текстуално име типа за серијализацију. Ово је важно јер сам TypeId није погодан као интероперабилан формат за трајно чување података.

Главне операције менаџера су:

- `register<T>(name)` - региструје тип компоненте по називу и прави складиште за тај тип ако не постоји,
- `add_component` - додаје компоненту уз креирање складишта по потреби,
- `get_storage_by_type` и `get_storage_mut_by_type` - типизован приступ уз *downcast*,
- `remove_all_components` - уклања везе између свих компоненти и датог ентитета,
- `serialize_all` и `deserialize_all` - серијализација / десеријализација свих регистрованих складишта компоненти.

Овај модел имплицира да је целокупан систем организован као скуп независних речника који се централизовано управљају преко менаџера компоненти.

3.4.4 Избор HashMap модела за складишта компоненти и његове алтернативе

Предности HashMap приступа за чување свих типова складишта компоненти у Rusty су:

- врло једноставно додавање / брисање компоненте,
- директно адресирање преко Entity идентификатора,
- мала сложеност кода ради лакшег разумевања и проширивања,
- лакша серијализација речника у JSON.

Мане произилазе из природе речника:

- лошија кеш локалност код масовног проласка кроз компоненте,
- додатни трошак хеш рачунања,
- мање погодан модел за SIMD / колонску обраду великих скупова података.

3.4.4.1 Архетипски приступ

Архетипски ЕКС организује ентитете у „табеле“ према тачној комбинацији компоненти коју поседују. Ако један ентитет има (Position, Velocity), а други (Position, Health), они припадају различитим архетипима.

Практична последица је да се компоненте чувају као колоне унутар архетипа, па су итерације система који обрађују фиксну комбинацију компоненти често веома брзе због добре кеш локалности [2].

Цена тог приступа је „миграција“ ентитета приликом додавања или брисања компоненте:

- промена сета компоненти мења архетип ентитета,
- ентитет мора да се премести у другу табелу,
- потребно је ажурирати мапирања и индексе.

Дакле, архетипски модел је веома добар за сценарије са честим итерацијама над групама ентитета су везани за исте компоненте, али је имплементационо сложенији од почетног HashMap решења.

3.4.4.2 Sparse set организација

Sparse set је компромис између једноставних мапа и строго архетипских табела. [2] Класична варијанта има:

- *dense*: компактну листу активних ентитета,
- *sparse*: речник *entity_id* -> индекс унутар *dense*,
- паралелан низ компоненти који прати *dense* редослед.

Тиме се добијају битне особине:

- брза провера постојања компоненте,
- компактна итерација по постојећим компонентама,
- релативно једноставно брисање (обично *swap-remove* техника).

У односу на архетип, *sparse set* је једноставнији за имплементацију и често веома добар у пракси. У односу на *HashMap*, углавном има бољу локалност при итерацији, али захтева више пажње око индекса и конзистентности структуре.

3.4.4.3 SoA колоне и хибридни модели

SoA (*Structure of Arrays*) чува свако поље у засебној колони (нпр. све *x* координате заједно), што додатно побољшава секвенцијално читање и *SIMD* обраду [2]. Међутим, *SoA* захтева пажљив дизајн АПИ-ја и често утиче на ергономију кода.

У индустрији су чести хибриди: архетип за често коришћене компоненте и ређи, динамични подаци у спољним структурама. *HashMap* као полазна тачка је оправдана јер чини изборе експлицитним, а каснију миграцију ка комплекснијем складишту могућом без рушења целог АПИ-ја.

3.5 event.rs: систем догађаја са редовима по типу

Модул *event.rs* уводи механизам асинхроне комуникације између система путем догађаја.

Event је *marker trait* (*Any* + *'static*), док *EventQueue<E>* представља *FIFO* (енг. *first in, first out*) ред конкретног типа догађаја заснован на *VecDeque<E>*.

```
pub struct EventQueue<E: Event> {
    events: VecDeque<E>,
}

pub fn push<E: Event>(&mut self, event: E) {
    self.register::();
    if let Some(queue) = self.get_queue_mut::() {
        queue.push(event);
    }
}
```

За складиштење више типова редова у једном менаџеру користе се *trait EventQueueTrait* и речник *HashMap<TypeId, Box<dyn EventQueueTrait>>* унутар *EventManager* структуре. Као код компоненти, гарантује се статичка типска безбедност на АПИ слоју и динамички *type erasure* унутар менаџера.

3.6 system.rs: системи и извршни редослед

system.rs дефинише *trait System* са методом *run(&mut self, world: &mut World)* и извршилац *SystemExecutor* који чува *Vec<Box<dyn System>>*.

```
pub trait System {
    fn run(&mut self, world: &mut World);
}
```



```
pub struct SystemExecutor {
    systems: Vec<Box<dyn System>>,
}

pub fn run(&mut self, world: &mut World) {
    for system in &mut self.systems {
        system.run(world);
    }
}
```

Извршавање је секвенцијално, редом којим су системи додати у вектор система, чиме се добија једноставан и детерминистички модел извршавања. Тестови у модулу експлицитно показују да редослед утиче на резултат (нпр. увећавање пре удвостручавања вредности није исто што и удвостручавање пре увећавања вредности).

У литератури и индустријским ЕКС системима често је и паралелно извршавање, где се системи извршавају конкурентно ако нема конфликтног приступа подацима. Такав модел може да повећа искоришћење вишејезгарних процесора, али захтева значајно сложеније праћење зависности писања и брисања података и често сложенији АПИ.

3.7 world.rs: интеграциони слој ЕКС језгра погона

World је централна фасада језгра и садржи сва три менаџера:

- EntityManager,
- ComponentManager,
- EventManager.

World игра излаже компактни АПИ и од корисника Rusty библиотеке сакрива комплексност унутрашњих подсистема.

```
pub fn destroy_entity(&mut self, entity: Entity) {
    self.components.remove_all_components(entity);
    self.entities.destroy(entity);
}

pub fn take_events<E: Event>(&mut self) -> Vec<E> {
    let mut events = Vec::new();
    if let Some(queue) = self.events.get_queue_mut:::<E>() {
        while let Some(event) = queue.pop() {
            events.push(event);
        }
    }
    events
}
```

3.7.1 Животни циклус ентитета у World-y

Метода destroy_entity прво позива remove_all_components, па тек онда entities.destroy(entity). Овај редослед је важан јер спречава да у складиштима остану сирочад (енг. *orphan*) компоненте за ентитет који више не постоји. У комбинацији са генерацијским моделом из entity.rs, добија се конзистентан механизам брисања и поновне употребе идентификатора без логичке колизије старих и нових ентитета.

3.7.2 Серијализација и десеријализација света

World дефинише интерну структуру WorldSaveData са кључним пољима:

- стање менаџера ентитета,
- серијализована складишта компоненти.

Снимање (save) и учитавање (load) реализовани су у JSON формату преко `serde_json` [11].

Након учитавања се позива `events.clear()`, чиме се избегава да се стари догађаји из претходног извршавања пренесу у ново учитано стање. Овиме стање света остаје поновљиво, а догађаји задржавају улогу транзијентног механизма комуникације, уместо да постану део трајног стања света.

3.8 Пример примене: текстуална борбена игра

Да би се видело како описана архитектура функционише у стварној употреби, развијена је текстуална игра у којој се Rusty ЕКС користи за једноставну потезну борбу играча против више непријатеља:

- ентитети представљају учеснике у игри (играч и непријатељи),
- компоненте носе податке (Name, Health, Damage, Defending, маркери Player и Enemy),
- догађаји (AttackEvent) представљају намеру акције,
- систем (DamageSystem) претвара догађаје у конкретну промену стања,
- SystemExecutor контролише редослед извршавања,
- World служи као фасадни АПИ кроз који се све оркестрира.

Следећи исечак кода показује почетну конфигурацију света и регистрацију система:

```
let mut world = World::new();

let player = world.create_entity();
world.add_component(player, Name("Hero".to_string()));
world.add_component(player, Player);
world.add_component(player, Health { hp: 45, max: 45 });
world.add_component(player, Damage { value: 7 });
world.add_component(player, Defending(false));

let mut executor = SystemExecutor::new();
executor.add_system(DamageSystem);
```

Овде се јасно види практична улога World АПИ-ја: `create_entity` враћа ентитет, док се `add_component` позива више пута да би се формирао комплетан „профил“ тог ентитета. Другим речима, понашање није везано за класну хијерархију, већ настаје композицијом компоненти.

Ток једног потеза реализован је кроз догађај и накнадно извршавање система:

```
world.push_event(AttackEvent {
    attacker: player,
    target: enemy,
    damage: dmg,
});

executor.run(&mut world);
```

Позив `push_event` смешта напад у ред догађаја, а тек у `executor.run` систем `DamageSystem` преузима све `AttackEvent` догађаје преко `take_events` и примењује последице на `Health` циља напада (играча). Тиме се раздваја опис самог догађаја од промене стања над циљем, у овом случају `Health` компонентом. Догађај само бележи да је дошло до напада, док се конкретна измена животних поена играча извршава накнадно у оквиру система који обрађују тај догађај.

Унутар система се користе `get_component` и `get_component_mut`: први за читање тренутног стања (име, улога, вредност напада), а други за безбедно ажурирање променљивих података (смањење `Health`). Тако библиотека задржава јасну границу између мутабилног и имутабилног приступа подацима у оквиру једног кадра извршавања.

Компонента `Name` је минимална *tuple struct* дефиниција (`struct Name(String);`) која служи за приказ у корисничком интерфејсу, без утицаја на саму механику борбе. Насупрот томе, `Player` је маркер-компонента без поља (`struct Player;`). Она је сигнал системима да је реч о ентитету који представља играча. Та разлика је јасна и у `DamageSystem`: `world.get_component::<Player>(attack.attacker).is_some()` одређује да ли се испишује порука у другом лицу („You strike...“) или у трећем („X hits you...“).

И непријатељи се уводе композиционо, кроз исти *АПИ*, без посебне класе. Уместо класе *Enemy*, у коду постоји скуп параметара (`enemy_attributes`) из ког се у петљи креира више ентитета:

```
let enemy_attributes = vec![
    ("Goblin", 12, 3, vec!["Slash", "Bite"]),
    ("Orc", 18, 5, vec!["Heavy Swing", "Headbutt"]),
    ("Necromancer", 22, 6, vec!["Shadow Bolt", "Bone Spike"]),
];

for (name, hp, dmg, _attacks) in &enemy_attributes {
    let e = world.create_entity();
    world.add_component(e, Name(name.to_string()));
    world.add_component(e, Enemy);
    world.add_component(e, Health { hp: *hp, max: *hp });
    world.add_component(e, Damage { value: *dmg });
    enemy_entities.push(e);
}
```

Овај део добро илуструје основну ЕКС предност: нов тип противника не захтева нову хијерархију типова, већ нову комбинацију компоненти и параметара. Компонента `Enemy` овде, као и `Player`, има улогу маркера који омогућава филтрирање и другачије понашање у логици игре.

Са становишта корисника библиотеке, логика игре се дефинише у једној `loop` петљи у функцији `main`, у оквиру које се ручно описује редослед интеракција (акција играча, акција непријатеља и покретање система). Са становишта *Rusty* погона, свака фаза ове петље само активира одговарајуће механизме обраде догађаја и система.

Поједностављено, ток изгледа овако:

```
loop {
    // 1) услови завршетка
    // 2) читање акције играча: attack / defend / quit
    // 3) push_event(...) за напад и executor.run(&mut world)
    // 4) ако је непријатељ жив, он шаље AttackEvent ка играчу
    // 5) system_executor.run(&mut world) за непријатељски потез
}
```

Са становишта архитектуре, кључно је да циклус игре не садржи логику умањења *HP*-а, већ само оркестрацију догађаја и позив система. Због тога је ток игре лако проширив: увођење нове акције, статус-ефекта или *AI* правила углавном се своди на додавање нових догађаја и / или система који их обрађују, док основна структура циклуса игре и модел оркестрације остају непромењени.

Глава 4

Имплементација ECS-a у rusty

Ово поглавље описује како су кључни ECS концепти конкретно реализовани у библиотеци `rusty`. Фокус је на структури `World`, моделу компоненти, механизму догађаја и организацији система, уз осврт на предности и ограничења изабраног решења.

4.1 Ентитети као лагани идентификатори

У ECS моделу ентитет не садржи доменску логику, већ представља идентификатор над којим се везују компоненте. Такав приступ омогућава да један ентитет по потреби добија или губи карактеристике у току извршавања, без промене типске хијерархије.

Практична последица је да животни циклус ентитета постаје једноставан:

- креирање новог ID-а,
- додавање иницијалног скупа компоненти,
- уклањање ентитета и придружених података када више није активан.

Овакав модел је посебно погодан за игре, где се број и тип објеката често динамички мењају током једне сцене.

4.2 Компоненте као чисти подаци

Компоненте су у пројекту осмишљене као структуре података без пословне логике. На пример, позиција, брзина, здравље, ознака видљивости или текстурни подаци представљају независне компоненте. Логика која их обрађује смештена је у системима.

Та подела има неколико важних последица:

- компоненте су једноставније за сериализацију,
- компоненте се лакше тестирају и поново користе,
- комбинацијом више компоненти добија се широк спектар понашања без наслеђивања.

У библиотеци је ово подржано генеричким API-јем, где се компоненте адресирају по типу. Типска безбедност језика Rust ту игра кључну улогу јер умањује ризик од грешака при приступу погрешном типу података.

Употреба Rust-а у овом слоју је посебно значајна јер компајлер намеће дисциплину која у C пројектима често остаје само договор у тиму. Када систем тражи `&mut` приступ некој компоненти, јасно је да у том делу кода не може постојати друга активна мутабилна референца на исте податке. Такав модел умањује читаву класу конкурентних и логичких грешака. Истовремено, у поређењу са C# приступом, овде је мање ослањања на runtime проверу и више гаранција се добија већ у фази компајлације.

4.3 Структура `World` као центар координације

`World` је централни координациони објекат ECS језгра. Његова улога је да обједини:

- управљање ентитетима,
- приступ складишту компоненти,
- размену догађаја између система.

```
pub fn create_entity(&mut self) -> Entity
pub fn destroy_entity(&mut self, entity: Entity)
pub fn add_component<T: Component>(&mut self, entity: Entity, component: T)
pub fn get_component<T: Component>(&self, entity: Entity) -> Option<&T>
pub fn get_component_mut<T: Component>(&mut self, entity: Entity) -> Option<&mut T>
```

Листинг 1: Основне операције над ECS светом

Из перспективе архитектуре, World представља једну тачку истине о тренутном стању симулације. Овај избор поједностављује развој и отежава појаву дуплираних извора стања, што је честа грешка у сложенијим системима.

Цена овог избора је што API мора пажљиво да се обликује како би остао практичан за употребу. У Rust-у то често значи да се ток података дизајнира у „фазама“: прво читање, затим израчунавање, па упис, уместо слободног мешања приступа као у мање строгим моделима. На први поглед ово делује као додатно оптерећење за програмера, али се у већим изменама показује корисним јер подстиче чисто раздвајање одговорности унутар система.

4.4 Складиште компоненти и регистар типова

Имплементација чува компоненте по типовима преко менаџера складишта. Кључна идеја је да сваки тип компоненте има сопствени storage, док је приступ апстрахован кроз заједнички интерфејс.

```
pub trait ComponentStorage {
    fn remove(&mut self, entity: Entity);
    fn serialize_storage(&self) -> serde_json::Value;
    fn deserialize_storage(&mut self, value: serde_json::Value);
}

pub struct ComponentManager {
    storages: HashMap<TypeId, Box<dyn ComponentStorage>>,
    names_to_typeids: HashMap<String, TypeId>,
}
```

Листинг 2: Апстракција складишта компоненти у rusty

Предност ове архитектуре је једноставна динамичка регистрација и приступ компонентама по типу. Недостатак је нешто већи overhead у односу на компактне архетајп структуре, посебно код екстремно великих сцена. Ипак, за потребе овог рада ово решење даје повољан баланс између јасноће имплементације и практичне употребљивости.

Овде је битно истаћи и ограничење које је свесно прихваћено: циљ није био да се одмах направи најбржа могућа ECS варијанта, већ да се покаже да и са безбедним апстракцијама може да се добије стабилан систем. У том смислу, Rust је послужио као филтер квалитета архитектуре: ако је неки део модела тешко изразити без unsafe, то је често сигнал да интерфејс или ток података треба додатно појаснити.

4.5 Системи и догађаји

Системи представљају извршне јединице које читају и мењају стање у World-у. Комуникација између система реализована је преко догађаја, чиме се смањује директна спрега. Уместо да системи међусобно позивају методе, један систем емитује догађај, а други га преузима у свом циклусу обраде.

```
pub fn push_event<E: 'static + Send>(&mut self, event: E)
pub fn take_events<E: 'static + Send>(&mut self) -> Vec<E>
```

Листинг 3: Основни API за рад са догађајима

Овим приступом се добија:

- боља декомпозиција логике по функционалним целинама,
- једноставнија замена или проширење система,

- лакше тестирање јер системи могу да се изолују по улазним догађајима.

Догађајни модел је посебно користан у контексту Rust-а јер смањује потребу да више система истовремено држи мутабилне референце на `World`. Уместо директног повезивања, системи размењују намере кроз догађаје, па се обрада може секвенцијално организовати уз јасно дефинисан ownership ток. Ово је пример како језичка ограничења не морају бити препрека, већ могу да воде ка чистијем архитектонском дизајну.

4.6 Пример из `text-game` пројекта

Текстуална игра илуструје практичан ECS ток у минималном окружењу. `AttackEvent` представља улазни подстицај, док `DamageSystem` обрађује догађај и ажурира компоненте као што су `Health` и `Defending`.

```
impl System for DamageSystem {
    fn run(&mut self, world: &mut World) {
        let attacks = world.take_events::<AttackEvent>();
        for attack in attacks {
            let mut damage = attack.damage;
            if is_defending(world, attack.target) {
                damage = (damage / 2).max(0);
            }
            if let Some(h) = world.get_component_mut::<Health>(attack.target) {
                h.hp = (h.hp - damage).max(0);
            }
        }
    }
}
```

Листинг 4: ECS обрада борбеног догађаја у `text-game`

Овај пример демонстрира да ECS модел није условљен графичким API-јем и да се једнако успешно примењује у системима који имају искључиво симулациони карактер.

4.7 Поређење са алтернативним имплементацијама

У литератури и пракси постоје две доминантне имплементационе стратегије:

- *sparse-set / hash-map оријентисан приступ*,
- *архетајп (archetype) приступ*.

Решење у овом раду је ближе првој групи. Његове главне предности су:

- лакша имплементација и разумевање,
- флексибилност у раду са динамичким типовима компоненти,
- погодност за образовне и прототипске пројекте.

Главна ограничења у односу на архетајп системе су:

- слабија локалност података,
- већи трошак итерације код масовне обраде,
- мање ефикасно пакетирање за SIMD/кеш оптимизације.

Ипак, за контекст овог рада (учење архитектуре, јасна анализа и практична демонстрација) ова ограничења су прихватљива. Најважније је да решење остаје довољно транспарентно да се могу јасно мерити ефекти будућих оптимизација. Другим речима, постављен је стабилан базни модел на који је могуће надограђивати перформансне технике без потпуног редизајна кода.

4.8 Инжењерске импликације за даљи развој

Тренутна имплементација обезбеђује стабилну основу за даљу еволуцију:

- увођење паралелног извршавања система на основу графа зависности,

- прелазак ка архетајп организацији компоненти у критичним сценаријима,
- проширење механизма догађаја приоритетним редовима и временским ознакама,
- унапређење сериализације ради снапшот/реплеј подршке.

Сви ови правци су компатибилни са постојећим API-јем ако се задржи јасна граница између јавног интерфејса и интерне реализације.

Глава 5

Рендеринг подсистем и избор wgpu

Рендеринг подсистем у оквиру `rusty` представља мост између ECS модела света и графичког уређаја. Његова примарна улога није само исцртавање објеката, већ и контролисано пресликавање компонентних података у формат погодан за GPU обраду.

5.1 Улога рендеринга у укупној архитектури

У ECS архитектури рендеринг је посматран као потрошач података из света. Он не дефинише правила симулације, већ визуализује њен резултат. Ова дистинкција је важна јер:

- логичка коректност игре остаје независна од графичког API-ја,
- исти симулациони модел може да ради и без графичког излаза,
- рендеринг се може мењати или унапређивати без нарушавања доменских система.

У практичном коду, рендеринг слој из `World`-а прикупља компоненте релевантне за приказ (позиција, спрајт, видљивост, редослед исцртавања), формира GPU инстанце и покреће цртање кадра.

5.2 `Renderer2D` као кључна апстракција

Структура `Renderer2D` обједињује иницијализацију графичког контекста и сам процес рендеровања. Њена одговорност обухвата:

- креирање `wgpu` инстанце и избор адаптера,
- добијање уређаја и командног реда,
- конфигурисање површине за приказ,
- припрему pipeline-а, бафера и bind група,
- извршавање `draw` позива у оквиру рендеринг пролаза.

```
pub fn render_world(&mut self, world: &World) -> Result<(), RenderError> {
    let batch = collect_visible_sprites(world);
    if batch.is_empty() {
        return self.render_batch(&[]);
    }
    let mut gpu_instances = Vec::with_capacity(batch.len());
    // мапирање ECS компоненти у GPU инстанце
    self.render_batch_with_order(&gpu_instances, &batch)
}
```

Листинг 5: Прелаз из ECS стања у GPU инстанце

Овај ток потврђује да је рендеринг у систему моделован као чиста трансформација подаци -> визуелна репрезентација.

Ова архитектонска одлука је значајна и за анализу избора Rust-а. Када се рендеринг третира као трансформациони слој, лакше је одржати строге ownership границе: симулација припрема податке, а рендерер их конзумира у јасно дефинисаном тренутку петље. На тај начин се избегавају нејасна преклапања одговорности која у C пројектима често воде ка имплицитним зависностима и сложеним баговима.

5.3 Основе `wgpu` модела

Библиотека `wgpu` имплементира WebGPU концепте у Rust екосистему и обезбеђује унификован приступ над више backend-а (Vulkan, Metal, DirectX, OpenGL у одређеним конфигурацијама). За овај рад то има директан инжењерски значај:

- апликација остаје преносива између платформи,
- графички модел је модеран и ближи савременој GPU пракси,
- API је типски и безбедносно строжи него код класичних C интерфејса.

Захваљујући томе, смањује се ризик од класе грешака које се у ниско-нивоским API-јима јављају услед неправилног управљања ресурсима.

Важно је нагласити да то не значи да Rust „магично“ уклања сву сложеност GPU програмирања. Напротив, концепти као што су pipeline стања, синхронизација команди, распоред бафера и животни век ресурса и даље захтевају пажљив дизајн. Разлика је у томе што компајлер и типски систем додатно помажу да број неконзистентних стања буде мањи, па се проблеми чешће откривају раније у развоју.

5.4 Зашто *wgri* уместо алтернатива

Избор *wgri* у овом пројекту условљен је потребом да решење истовремено буде:

- довољно ниско-нивоско за учење реалног рендеринг тока,
- довољно безбедно и читљиво за академску анализу,
- довољно преносиво за различита окружења.

У поређењу са алтернативама:

- OpenGL + *glum* може бити једноставнији за почетак, али мање прати модерни *explicit* модел ресурсног управљања,
- директни Vulkan/DirectX/Metal нуде максималну контролу, али уз значајно већу сложеност и већу вероватноћу грешака,
- виши *framework*-ови (нпр. готови *game engine* слојеви) убрзавају прототипирање, али прикривају критичне детаље који су важни за циљ овог рада.

Стога *wgri* представља рационалан компромис: задржава се практична дубина, а избегава се непотребна сложеност.

У контексту поређења са C и C#, овај компромис је посебно видљив. C + Vulkan даје највећу контролу, али је број детаља који програмер мора ручно да одржава веома висок. C# решења вишег нивоа обично омогућавају брже прототипирање, али често сакривају кључне детаље GPU пута који су за овај рад методолошки важни. Rust + *wgri* је одабран јер дозвољава да се ти детаљи уче и анализирају, а да се истовремено минимизује ризик од класичних *memory-safety* грешака у апликативном коду.

5.5 Пут од ECS компоненти до *draw* позива

Кључни кораци у сваком кадру су:

1. избор видљивих ентитета,
2. формирање инстансних података (позиција, скала, ротација, UV/текстура),
3. упис података у GPU бафере,
4. подешавање pipeline стања,
5. извршавање инстансног цртања.

Овакав приступ смањује број *draw* позива када више објеката дели исти pipeline и текстурне ресурсе, што је важно за стабилан FPS при већим сценама.

5.6 Шејдерски слој и ресурсни модел

Рендеринг подсистем користи шејдере као формалну дефиницију трансформације геометрије и боје. Иако је конкретна логика шејдера у пројекту сведена на 2D случај, архитектонски шаблон је скалабилан:

- вршни шејдер дефинише позиционирање у простору кадра,
- фрагментни шејдер управља читањем текстуре и бојом пиксела,

- bind групе раздвајају униформе, текстуре и саплере у јасне ресурсне блокове.

Та дисциплина у раду са ресурсима је директно усклађена са циљевима модерних API-ја: предвидивост, експлицитност и лакша оптимизација.

5.7 Управљање редоследом исцртавања

У 2D контексту редослед исцртавања (z-order, layer) је функционално критичан. Рендерер зато приликом формирања batch-а води рачуна о редоследу објеката. Концептуално, то омогућава исправно преклапање спрајтова без ослањања на сложени 3D depth pipeline.

Успостављањем јасног правила редоследа, визуелни резултат постаје стабилан и репродуктиван, што је важно и за тестирање и за бенчмарк поређења.

Репродуктивност је у овом раду значајна не само због визуелне коректности, већ и због валидности мерења. Ако редослед приказа варира без јасног правила, перформансне разлике између покретања могу бити последица неочекиваног стања, а не стварне промене у архитектури. Зато су детерминисана правила припреме batch-а и draw редоследа важан део методологије евалуације.

5.8 Перформансне импликације

Перформансе рендеринга зависе од више фактора:

- броја инстанци у кадру,
- учесталости ажурирања бафера,
- броја прелаза стања pipeline-а,
- трошка синхронизације између CPU и GPU стране.

У контексту овог пројекта, најважнији ефекат има број активних објеката и квалитет batch-овања. Зато је у rendering-benchmark апликацији фокус стављен на контролисано повећање броја објеката уз праћење FPS метрике.

Поред тога, релевантно је посматрати и карактер пада перформанси. Линеарни пад FPS-а са растом оптерећења обично указује на предвидиво понашање система, док нагли нелинеарни прекиди могу бити сигнал да је достигнут праг капацитета конкретне компоненте (нпр. write path ка GPU баферима или учесталост смене pipeline стања). Такав тип анализе је важан јер помера фокус са „колико FPS има“ на „зашто се FPS мења“.

5.9 Ограничења тренутне имплементације

Иако рендеринг подсистем испуњава пројектне циљеве, уочљива су и ограничења:

- примарно је усмерен на 2D спрајт сценарио,
- не садржи напредне технике (instanced texture atlases, culling стратегије, мултипас ефекте),
- не обухвата дубљу аутоматску оптимизацију распореда ресурса.

Ово, међутим, не представља архитектонску препреку за даљи развој. Напротив, постојећи дизајн је довољно чист да омогући постепено додавање сложенијих функционалности без ломљења јавног интерфејса.

Глава 6

Рендеринг бенчмарк и евалуација перформанси

Поред архитектонске исправности, играчки погон мора да испуни и практични захтев: стабилно понашање под реалним оптерећењем. Због тога је у оквиру пројекта реализована посебна апликација `rendering-benchmark`, чији је циљ квантификација односа између броја објеката у сцени и постигнуте брзине рендеринга.

6.1 Циљ и улога бенчмарка

Основна улога бенчмарка је да обезбеди контролисано окружење у коме се може пратити понашање система при променљивом оптерећењу. За разлику од класичних `unit` тестова, који проверавају функционалну исправност, бенчмарк даје увид у перформансне трендове и оперативне границе решења.

Конкретно, бенчмарк омогућава:

- интерактивно мењање броја рендерованих објеката,
- праћење FPS метрике у реалном времену,
- посматрање стабилности кадра при динамичким променама сцене,
- идентификацију тачака у којима долази до значајнијег пада перформанси.

6.2 Конструкција сценарија

Сценарио је дизајниран тако да садржи велики број 2D објеката (до 50_000) који пролазе кроз циклус ажурирања и рендеровања у сваком кадру. На тај начин се истовремено оптерећују:

- ECS део (ажурирање стања и компоненти),
- припрема рендеринг `batch-a`,
- GPU извршавање `draw` позива.

Оваква поставка је погодна за испитивање *end-to-end* понашања, јер не посматра само један изолован подсистем већ целокупан пут од података до приказа.

Посебна вредност оваквог сценарија је у томе што омогућава да се архитектонске одлуке оцењују у реалистичном току, а не само кроз изоловане микротестове. На пример, и ако је појединачна операција над компонентама брза, укупни резултат може бити ограничен трошком формирања `batch-a` или уписа инстансних података у GPU бафере. Зато је у овом раду приоритет дат интегралном мерењу, које боље одражава кориснички осећај „глаткоће“ апликације.

6.3 Механизам мерења FPS-a

Кључна метрика у бенчмарку је FPS (`frames per second`), која се рачуна у кратким временским прозорима ради ублажавања тренутних осцилација.

```
if self.fps_timer >= 0.25 {
    self.fps = self.fps_frames as f32 / self.fps_timer;
    self.fps_timer = 0.0;
    self.fps_frames = 0;
    self.title_dirty = true;
}
```

Листинг 6: Периодично израчунавање FPS вредности

Интервал од 0.25 секунди представља практичан компромис: довољно је кратак за брзу повратну информацију, а довољно дуг да смањи шум тренутних варијација.

Методолошки, овај избор прозора мерења смањује ризик да закључци буду донети на основу једнокадровских осцилација (нпр. кратки скокови услед OS scheduler-а или периодичних графичких операција). Иако FPS није једина релевантна метрика, управо у контексту интерактивног 2D сценарија он даје интуитивно разумљив показатељ односа између оптерећења и визуелне стабилности.

6.4 Интерактивна контрола оптерећења

Бенчмарк је реализован тако да корисник може да мења број објеката и параметре сцене током рада апликације. То омогућава визуелно и квантитативно праћење последица сваке промене, без поновног покретања програма.

У методолошком смислу, ова интерактивност је важна јер:

- омогућава брзо мапирање перформансне криве,
- олакшава уочавање прагова деградације,
- подржава квалитативно посматрање fluidity осећаја уз нумеричку метрику.

Додатно, интерактивна контрола омогућава да се у кратком времену тестирају различите хипотезе о уским грлима. На пример, ако се FPS знатно мења већ при малом повећању броја објеката, то може упутити на overhead управљања подацима, а не нужно на GPU лимит. Ако пад постаје изражен тек при већим вредностима, вероватније је да је доминантан трошак на рендеринг страни. Овакво иницијално дијагностичко читање није замена за профилисање, али помаже да се профилисање усмери на праве делове система.

6.5 Предложена методологија спровођења мерења

Да би резултати били репродуктивни и релевантни, препоручује се следећи поступак:

1. покренути апликацију у истим условима система (без додатног оптерећења),
2. стабилизовати сцену на почетном броју објеката,
3. повећавати број објеката у унапред дефинисаним корацима,
4. за сваки корак бележити просечан FPS у временском интервалу,
5. поновити мерење више пута и анализирати средњу вредност.

Ова процедура минимизује утицај случајних варијација и омогућава прављење конзистентних поређења између различитих верзија кода.

За већу репродуктивност препоручљиво је бележити и пратеће услове мерења:

- верзију драјвера и графичког backend-а,
- резолуцију прозора и режим приказа,
- активне апликације у позадини,
- верзију кода и commit идентификатор.

На тај начин резултати добијају истраживачку вредност, јер је касније могуће поновити експеримент у што сличнијим условима и проверити да ли се трендови задржавају.

6.6 Тумачење резултата

Резултате FPS мерења не треба посматрати апсолутно, већ у контексту:

- хардверске конфигурације,
- резолуције приказа,
- активних графичких backend-а,
- укупног архитектонског дизајна.

Упркос томе, трендови су веома информативни. Ако повећање броја објеката доводи до приближно линеарног пада FPS-а, то указује на очекивану скалабилност унутар модела. Нагли нелинеарни падови могу сигнализирати уска грла у batching-у, упису бафера или организацији података.

У пракси је корисно разликовати три режима понашања:

1. стабилан режим — промена оптерећења има умерен и предвидив утицај,
2. прелазни режим — систем почиње да губи конзистентност времена кадра,
3. деградирани режим — FPS пада испод циљног прага за употребљивост.

Ова класификација помаже да се резултати не тумаче бинарно (добро/лоше), већ као континуум који директно води ка приоритетима оптимизације.

6.7 Веза са архитектурним одлукама

Бенчмарк директно рефлектује последице раније описаних одлука:

- HashMap оријентисано складиште компоненти утиче на кеш локалност,
- секвенцијално извршавање система ограничава коришћење више језгара,
- квалитет batch-a и број draw позива утичу на GPU ефикасност.

Због тога бенчмарк није само алат за мерење, већ и дијагностички инструмент који повезује архитектуру и оперативно понашање.

У контексту избора Rust-a, ово повезивање има додатну вредност. Циљ није био да се докаже апсолутна супериорност једног језика, већ да се провери да ли је у реалном сценарију могуће одржати прихватљиве перформансе уз строгу безбедносну дисциплину и без сопственог unsafe кода. Добро понашање бенчмарка у ширем опсегу оптерећења подржава тезу да је такав приступ практично одржив за пројекте овог обима.

6.8 Ограничења бенчмарк приступа

Иако користан, актуелни бенчмарк има ограничења која треба експлицитно навести:

- фокусиран је на један тип сцене (2D спрајтови),
- не издваја експлицитно CPU и GPU време по кадру,
- не обухвата поређење више рендеринг стратегија у истим условима,
- не укључује аутоматски експорт резултата у табеларном облику.

Ограничења су прихватљива у контексту основног циља рада, али указују на јасан простор за методолошко проширење.

Такође, ограничења указују и на разлику између академског и индустријског фокуса. За потребе овог рада било је важније показати архитектонску кохерентност и контролисану евалуацију него извршити максимални број ниско-нивоских оптимизација. У индустријском контексту, следећи корак би вероватно укључио детаљније профилисање CPU/GPU пута и аутоматизовану анализу временских серија.

6.9 Предлози за унапређење евалуације

За наредне итерације пројекта препоручује се:

- увођење више профила сцена (различите густине и величине објеката),
- додавање раздвојених метрика (update time, render time, frame time),
- аутоматизација серије покретања и чување резултата у CSV/JSON,
- визуелизација трендова графицима ради лакшег поређења,
- поређење HashMap и архетајп варијанте на истом workload-у.

Ови кораци би омогућили прецизнију квантификацију ефеката архитектонских промена и донели јачу емпиријску подлогу за будуће оптимизације.

У овом раду анализирана је и документована имплементација пројекта Rust-ECS, са посебним фокусом на избор ECS архитектуре, њену практичну примену у rusty библиотеци и везу са два демонстрациона пројекта: `text-game` и `rendering-benchmark`.

Кроз структуру рада показано је да се кључне архитектонске одлуке могу пратити од теоријске мотивације, преко имплементационих детаља, до емпиријске провере у контролисаном сценарију. Тај континуитет је био важан методолошки циљ, јер омогућава да закључци не остану на нивоу општих тврдњи, већ да буду ослоњени на конкретне ефекте у коду и у понашању система.

Показано је да ECS приступ доноси јасно раздвајање података и логике, једноставније проширење функционалности и добру основу за тестирање. У конкретној имплементацији то се огледа у централној структури `World`, типски раздвојеним складиштима компоненти и систему догађаја који смањује спрегу између делова програма.

Посебан допринос рада је и аргументовано образложење избора Rust-а као имплементационог језика. Пројекат је послужио као практичан тест да ли је могуће изградити употребљив ECS и рендеринг подсистем без увођења `unsafe` кода у апликативном слоју. У датом обиму и циљевима рада, показано је да је то могуће, уз прихватљив однос између сложености развоја и добијене поузданости.

У поређењу са C приступом, Rust је у овом пројекту смањио простор за класичне `memory-safety` грешке и померио фокус на дисциплинованији дизајн интерфејса. У поређењу са C# приступом, цена је нешто већи иницијални напор у моделовању власништва и приступа подацима, али уз већу контролу над ресурсима и предвидивије понашање у сценаријима који су ближи системском програмирању. Практично, то значи да је „цимање“ израженије на почетку, али да каснији развој добија на стабилности.

Рендеринг подсистем реализован је преко `wgpu` библиотеке, што обезбеђује преносивост и модеран GPU модел уз прихватљиву сложеност. Истовремено, бенчмарк сценарио показује како се архитектонске одлуке одражавају на перформансе у условима променљивог оптерећења.

Евалуациони део је показао да су трендови перформанси довољно информативни за дијагностику уских грла и приоритизацију даљих оптимизација. Иако бенчмарк није замена за дубинско профилисање, његова вредност је у томе што даје брз, репродуктиван увид у везу између архитектуре и оперативног понашања.

Главна ограничења тренутног решења су `HashMap` организација компоненти и секвенцијално извршавање система, што представља простор за даља унапређења. Као природан наставак развоја издвајају се аркетајп складиште, паралелизација, као и проширење бенчмарк сценарија и комплетнија анализа метрика.

У ширем смислу, резултат рада показује да је за академски и прототипски контекст оправдано кренути од архитектонски чистог и безбедног решења, па тек затим уводити агресивније оптимизације где мерења покажу да су неопходне. Такав редослед смањује ризик од преране сложености и омогућава да свако унапређење буде аргументовано мерљивим ефектом.

Списак коришћених скраћеница

Скраћеница	Опис
АПИ	Application Programming Interface (програмски интерфејс)
ЕКС	Entity Component System (ентитет-компонента-систем архитектура)
JSON	JavaScript Object Notation (формат за размену података)
UUID	Universally Unique Identifier

Списак коришћених појмова

Појам	Објашњење
Ентитет	Логички објекат у ECS моделу који представља идентификатор без директно уграђеног понашања.
Компонента	Структура података која описује један аспект стања ентитета (нпр. позиција, спрајт, здравље).
Систем	Јединица логике која обрађује скупове компоненти и примењује правила симулације.
Свет (world)	Централна структура која управља ентитетима, компонентама, системима и током догађаја у оквиру апликације.
Рендеринг pipeline	Скуп фаза и стања којима се ECS подаци трансформишу у графички приказ на екрану.
Batching	Груписање више објеката у мањи број draw позива ради боље ефикасности рендеринга.
Инстансни рендеринг	Техника исцртавања више визуелно сличних објеката једним draw позивом уз различите инстансне параметре.
Шејдер	Програм који се извршава на GPU-у и дефинише како се обрађују геометрија и боја током рендеринга.
Бенчмарк	Контролисан сценарио мерења перформанси којим се процењује понашање система под различитим оптерећењима.
Скалабилност	Способност система да задржи прихватљив ниво перформанси при расту броја објеката и сложености сцене.
Type erasure	Сакривање конкретног типа иза заједничког интерфејса.
Runtime type identification	Препознавање типа током извршавања програма.
Downcast	Повратак са општег динамичког на конкретан тип.
Архетипски ECS	Груписање ентитета по истој комбинацији компоненти.
Sparse set	Комбинација dense и sparse низа за брзо чланство и итерацију.
SoA (Structure of Arrays)	Чување података по колонама уместо по објектима.
FIFO	Редослед „први ушао, први изашао“.
Фасада	Јединствен API који сакрива унутрашњу сложеност подсистема.

Биографија

У основној школи правио видео игре користећи платформу Sploder. Касније током основне школе учио програмирање користећи Scratch и затим копирајући код снимака са платформе YouTube.

У Шабачкој гимназији похађао смер за ученике са посебним способностима за рачунарство и информатику.

Од 2022. до 2026. похађао основне академске студије софтверског инжењерства и информационих технологија на Факултету техничких наука у Новом Саду.

Након прве године основних студија одрадио две неплаћене праксе:

Литература

- [1] R. Lord, „Evolve your hierarchy“. Приступљено: 30. Мај 2026. [На Интернету]. Доступно на <https://www.richardlord.net/blog/ecs/what-is-an-entity-framework.html>
- [2] R. Fabian, *Data-Oriented Design*. The Pragmatic Programmers, 2020.
- [3] Unity Technologies, „Unity's Data-Oriented Technology Stack (DOTS)“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://unity.com/dots>
- [4] Unity Technologies, „Entities overview | Entities | 1.0.16“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://docs.unity3d.com/Packages/com.unity.entities@1.0>
- [5] Epic Games, „Gameplay Framework in Unreal Engine“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-in-unreal-engine?lang=en-US>
- [6] Bevy Foundation and contributors, „bevyengine/bevy: A refreshingly simple data-driven game engine built in Rust“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://github.com/bevyengine/bevy>
- [7] Bevy contributors, „Crate bevy - docs.rs“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://docs.rs/bevy>
- [8] S. Klabnik и C. Nichols, „The Rust Programming Language: Unsafe Rust“. Приступљено: 29. Мај 2026. [На Интернету]. Доступно на <https://doc.rust-lang.org/book/ch20-01-unsafe-rust.html>
- [9] Bevy contributors, „Bevy entity documentation“. Приступљено: 01. Јуни 2026. [На Интернету]. Доступно на <https://docs.rs/bevy/latest/bevy/prelude/struct.Entity.html>
- [10] E. Tryzelaar и D. Tolnay, „Serde“. Приступљено: 01. Јуни 2026. [На Интернету]. Доступно на <https://serde.rs/>
- [11] D. Tolnay, „serde_json - docs.rs“. Приступљено: 01. Јуни 2026. [На Интернету]. Доступно на https://docs.rs/serde_json/latest/serde_json/